Hardware Assisted and
Blockchain Empowered Security Lab

Laboratorio – Programmazione del calcolo quantistico

Franco Cirillo
University of Salerno
fracirillo@unisa.it

Tecnologie quantistiche per la sicurezza

Qiskit

"Qiskit is the world's most popular and performant **software stack** for quantum computing and algorithms research. **Build, optimize, and execute** quantum workloads at scale."

13M

Qiskit downloads

69%

Developers prefer Qiskit as primary SDK, according to the Unitary Foundation

83x

Faster transpilation time, according to Benchpress toolkit

7K+

Dependent projects

Qiskit

Quantum information science

Power tools for designing quantum experiments and new capabilities.

Learn more ↓

Heterogeneous orchestration

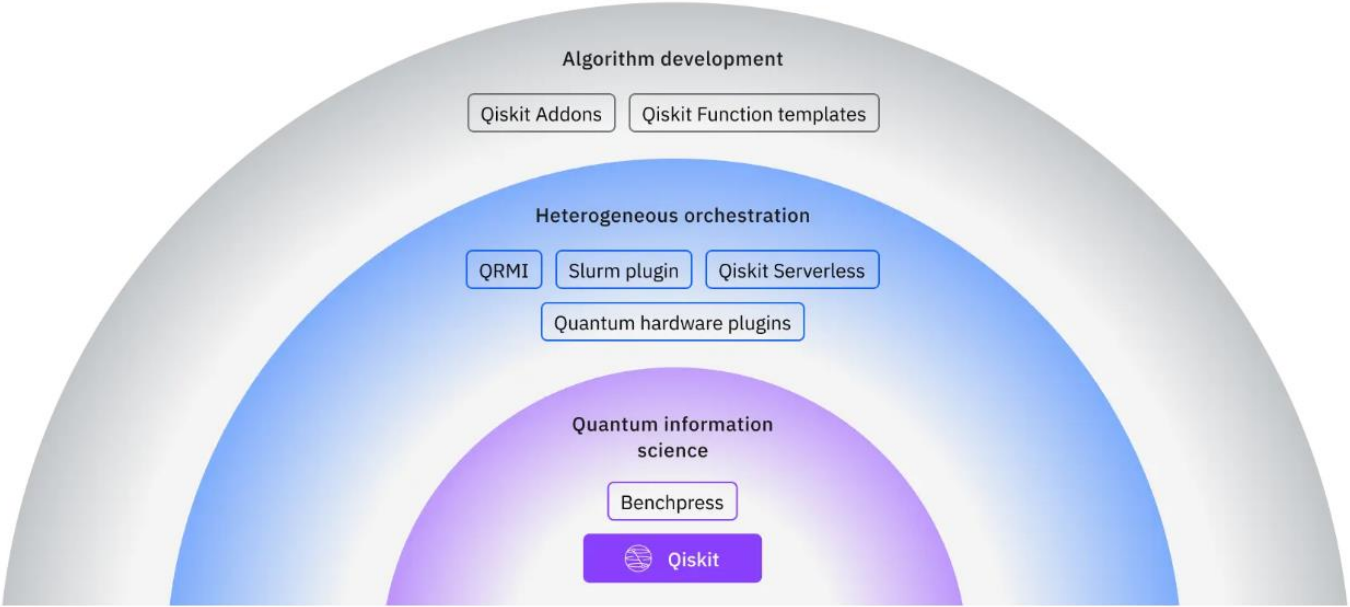
Qiskit plugins support orchestratic and classical resources.

Learn more ↓

Algorithm development

Tools map problems to circuits anc them for execution.

Learn more ↓



Quantum information science toolkit

Qiskit is an open-source SDK for working with quantum computers at the level of extended quantum circuits, operators, and primitives.

https://github.com/Qiskit/qiskit
Latest version 2.3.0

https://quantum.cloud.ibm.com/docs/

Build quantum circuits from quantum gates

Compile circuits for execution

Use primitives to build and execute experiments

Visualize circuits and analyze results

Qiskit plugins

Qiskit quantum hardware plugins

Qiskit is open-source and backend-agnostic, supporting quantum execution across a broad range of popular hardware providers.

View all →

Amazon Braket • Azure Quantum • IQM • Rigetti • Quantinuum • Superstaq • AQT • Alice & Bob •

Quantum resource management

The quantum resource management interface (QRMI) standardizes and simplifies control of quantum resources.

View repo

Slurm plugin

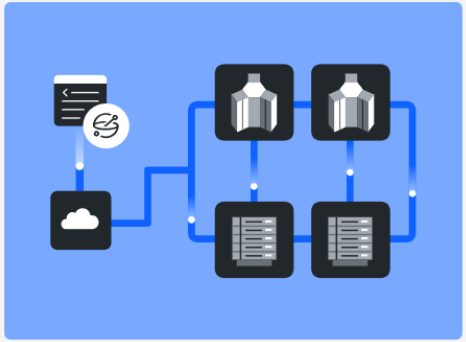
This plugin allows you to manage quantum resources using the Slurm workload manager.

View repo

Qiskit Serverless

A simple interface to run workloads across quantum-classical resources, supporting multi-cloud and supercomputing use cases.

View repo



Qiskit plugins

https://www.ibm.com/quantum/ecosystem

TorchQuantum

Repo

A PyTorch-centric hybrid classical-quantum dynamic neural networks framework.

1605 Community-maintained

QML

Qiskit Machine Learning

Repo Documentation

Use flexible building blocks, such as quantum kernels and neural networks, to create quantum machine learning algorithms.

943 Community-maintained

QML

Aer

Repo Documentation

High-performance quantum computing simulators with realistic noise models.

637 IBM-maintained

provider

Mitiq

Repo

Mitiq is a Python toolkit for implementing error mitigation techniques on quantum computers.

414 Community-maintained

error mitigation

Qiskit Nature

Repo Documentation

Model and solve problems in physics, chemistry, material science, and biology by using quantum simulations.

372 Community-maintained

chemistry physics

Qiskit Finance

Repo Documentation

Includes uncertainty components for stock/securities problems, Ising translators for portfolio optimizations, and data providers.

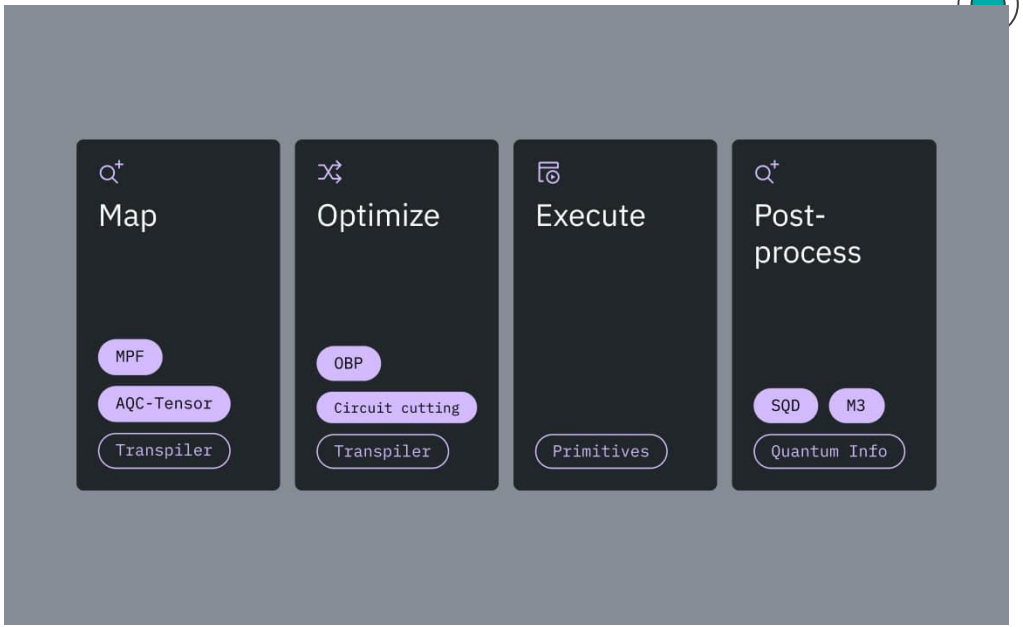
304 Community-maintained

finance

Qiskit addons

Gli add-on Qiskit sono una **raccolta di funzionalità** di ricerca che consentono la scoperta di algoritmi su scala industriale.

Molti di questi componenti aggiuntivi sono alimentati dal pacchetto **qiskit-addon-utils** .



Qiskit

Optimization

Explore quantum approaches to combinatorial problems that challenge existing classical methods.

Simulation

Use quantum and HPC to simulate complex systems in nature across physics and chemistry domains.

Quantum machine learning

Boost classical machine learning and uncover new solutions by leveraging quantum kernels.

Partial differential equations

Simulate nonlinear dynamical systems, such as fluid flow dynamics, using quantum variational methods.

Algorithms and applications

Problem mapping | Circuit functions | Workflow templates | Application functions

Qiskit

Preparing circuits & operators | Transpiling
Execution models | Error mitigation
Sample-based algorithms | Code assistants

IBM Quantum Platform

Execute on QPUs | Account setup
Administration | Workload scheduling
Orchestrating quantum-classical workloads

Qiskit overview

Il nome "Qiskit" è un termine generico che si riferisce a **una raccolta di software** per l'esecuzione di programmi su computer quantistici.

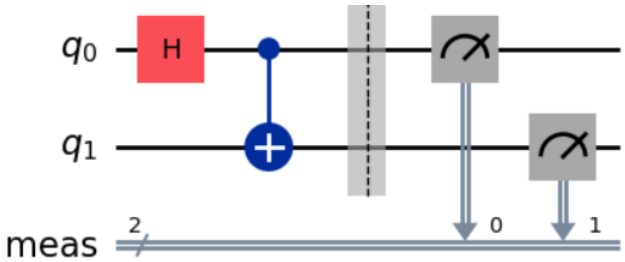
Tra questi strumenti software spiccano l'open-source Qiskit SDK e l'ambiente di runtime (a cui si accede tramite Qiskit Runtime) attraverso il quale è possibile eseguire carichi di lavoro sulle unità di elaborazione quantistica (QPU) di IBM®.

Inoltre, molti progetti open-source fanno parte del più ampio **ecosistema** Qiskit. Questi strumenti software non fanno parte di Qiskit stesso, ma si interfacciano con Qiskit e possono fornire preziose funzionalità aggiuntive.

Get Started

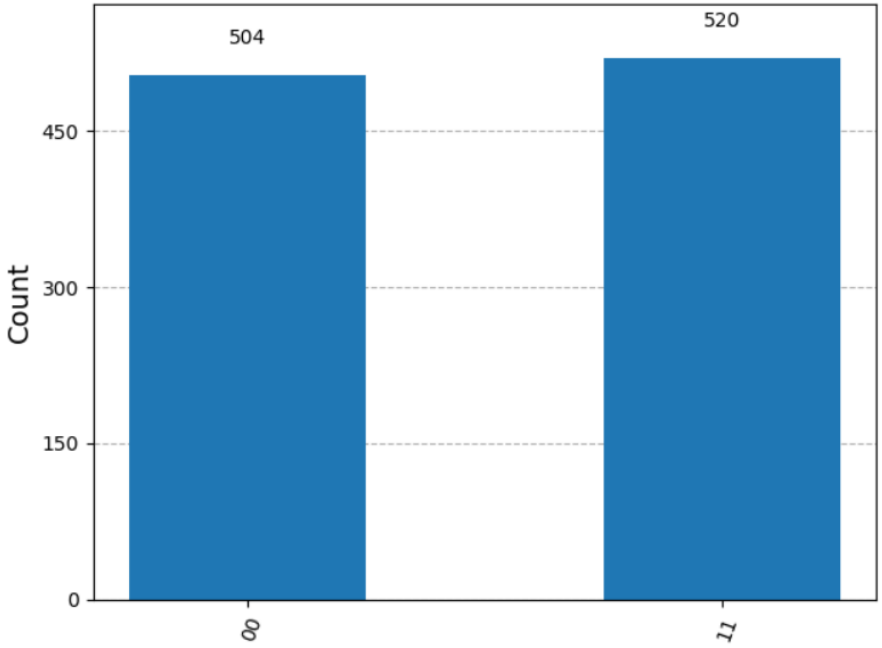
```
!pip install qiskit
!pip install 'qiskit[visualization]'
!pip install matplotlib

from qiskit import QuantumCircuit
from qiskit.primitives import StatevectorSampler

qc = QuantumCircuit(2)
qc.h(0)
qc.cx(0, 1)
qc.measure_all()

sampler = StatevectorSampler()
result = sampler.run([qc], shots=1024).result()
print(result[0].data.meas.get_counts())
qc.draw("mpl")
```

```
{'11': 520, '00': 504}
```



Get Started

```
import matplotlib.pyplot as plt
from qiskit.visualization import plot_histogram

counts = result[0].data.meas.get_counts()
plot_histogram(counts)

plt.show()
```



Esercizio

Provate a modificare il codice per vedere come influisce sui risultati.

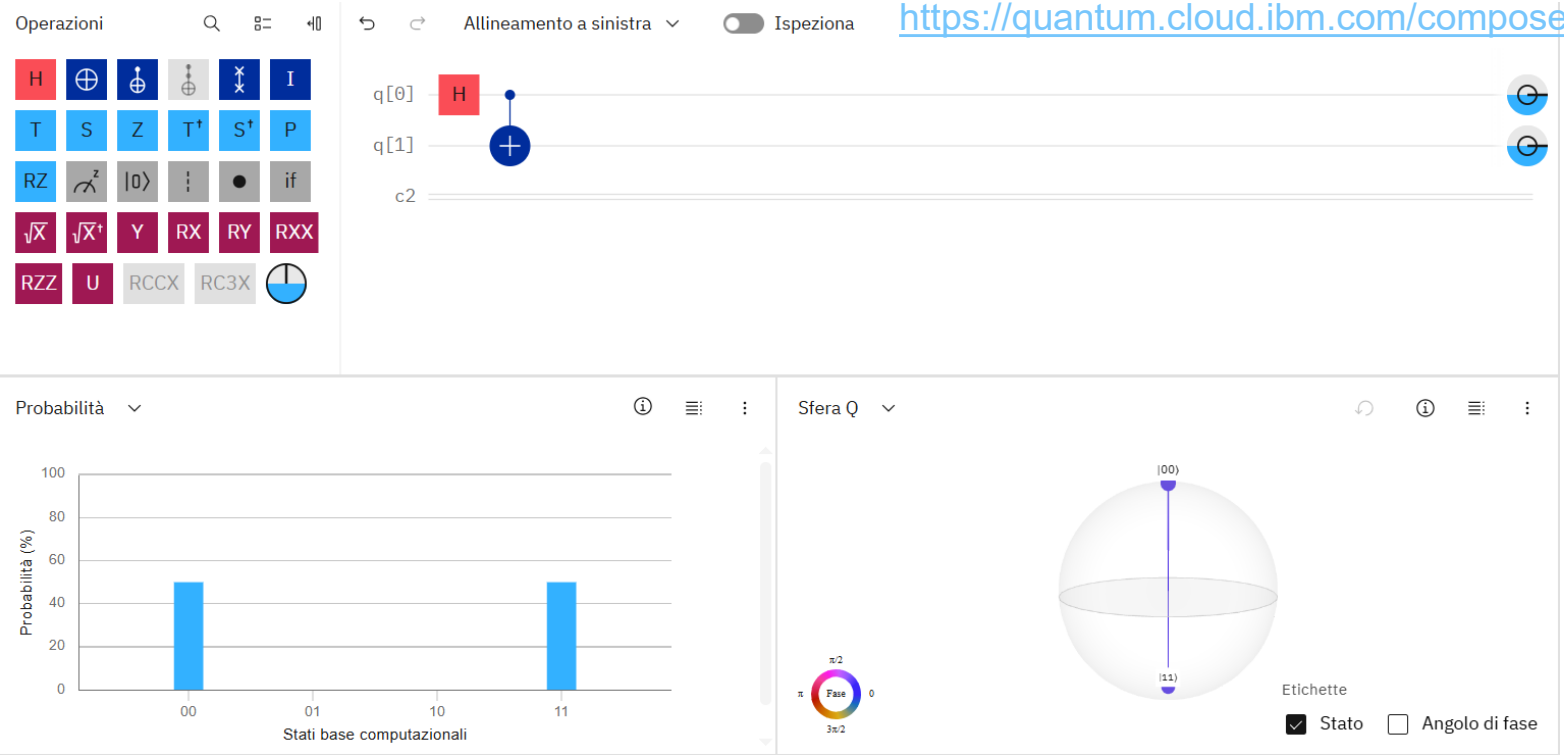
Ad esempio:

- Aggiungete un terzo qubit passando a QuantumCircuit(3), e aggiungete una seconda porta CX con qc.cx(1,2).

- Le misure dovrebbero poi cambiare in 000 e 111, il che significa che tutti e tre questi qubit sono stati entangled.

- Osservate il cambiamento dei risultati aggiungendo qc.x(1) alla fine del circuito.

Composer


Operazioni
Allineamento a sinistra
Ispeziona
https://quantum.cloud.ibm.com/composer
H
I
T
S
Z
T†
S†
P
RZ
|0⟩
if
√X
√X†
Y
RX
RY
RXX
RZZ
U
RCCX
RC3X
q[0]
q[1]
c2
Probabilità
Probabilità (%)
100
80
60
40
20
0
00
01
10
11
Stati base computazionali
Sfera Q
|00⟩
|11⟩
π/2
π
Fase
0
3π/2
Etichette
Stato
Angolo di fase

O' Reilly

https://oreilly-qc.github.io/

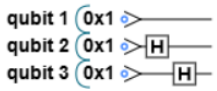
Run Program | Ex 3-1: Separable q... | QCEngine

```
// Programming Quantum Computers
//   by Eric Johnston, Nic Harrigan and Mercedes Gimeno-Segovia
//   O'Reilly Media

// To run this online, go to http://oreilly-qc.github.io?p=3-1

qc.reset(3);
qc.write(0);
var qubit1 = qint.new(1, 'qubit 1');
var qubit2 = qint.new(1, 'qubit 2');
var qubit3 = qint.new(1, 'qubit 3');
qubit2.had();
qubit3.had();
```

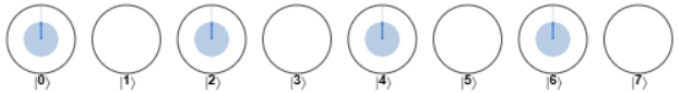
Source code on Github

QCEngine | Qiskit | OpenQASM | Q# | Cirq

Program circuit



Circle notation



1: Circuit design

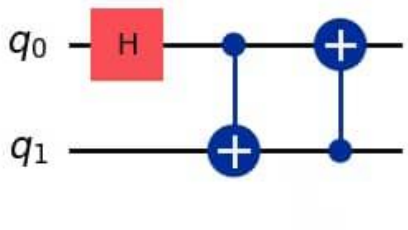
- Standard gates: H, X, CX, CX CCnot...
- N-local circuits
- Data-encoding circuits
- Time-evolution circuits
- Benchmarking and complexity-theory circuits
- Arithmetic circuits

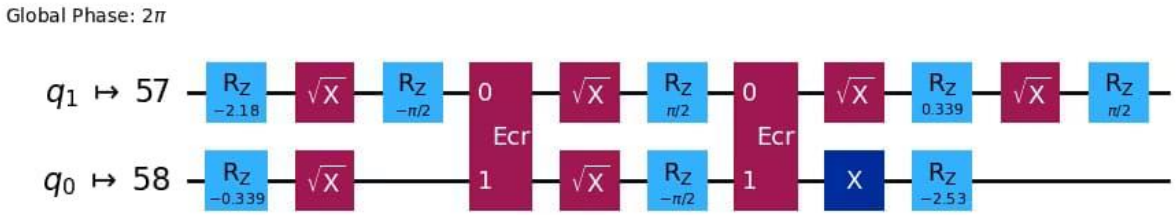
```
from qiskit.circuit.library import zz_feature_map

features = [0.2, 0.4, 0.8]
feature_map =
zz_feature_map(feature_dimension=len(features))

encoded = feature_map.assign_parameters(features)
encoded.draw("mpl")
```

2: Transpilation

È il processo di riscrittura di un determinato circuito di input per adattarlo alla topologia di uno specifico dispositivo quantistico e ottimizzare le istruzioni del circuito per l'esecuzione su computer quantistici rumorosi.

La trasposizione lo trasforma in modo da utilizzare solo le istruzioni disponibili su un backend prescelto e ottimizza tali istruzioni per ridurre al minimo gli effetti del rumore.

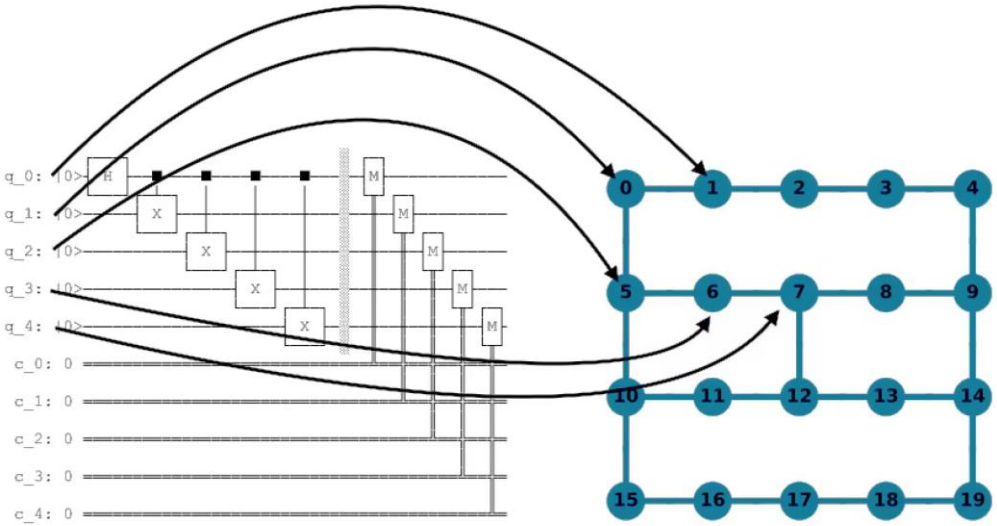




2: Transpilation

È il processo di riscrittura di un determinato circuito di input per adattarlo alla topologia di uno specifico dispositivo quantistico e ottimizzare le istruzioni del circuito per l'esecuzione su computer quantistici rumorosi.

La trasposizione lo trasforma in modo da utilizzare solo le istruzioni disponibili su un backend prescelto e ottimizza tali istruzioni per ridurre al minimo gli effetti del rumore.



2: Transpilation

```
import matplotlib.pyplot as plt
from qiskit import QuantumCircuit
from qiskit_ibm_runtime.fake_provider import FakeAuckland, FakeWashingtonV2
from qiskit.transpiler import generate_preset_pass_manager

backend = FakeAuckland()
ghz = QuantumCircuit(15)
ghz.h(0)
ghz.cx(0, range(1, 15))
depths = [ ]
for seed in range(100):
    pass_manager = generate_preset_pass_manager(
        optimization_level=1,
        backend=backend,
        layout_method="trivial",  # Fixed layout mapped in circuit order
        seed_transpiler=seed,  # For reproducible results
    )
    depths.append(pass_manager.run(ghz).depth())
plt.figure(figsize=(8, 6))
plt.hist(depths, align="left", color="#AC557C")
plt.xlabel("Depth", fontsize=14)
plt.ylabel("Counts", fontsize=14)
```

3: Execution

Primitive

Inizialmente Qiskit ha introdotto *backend.run*, una prima astrazione per accedere alle QPU nel cloud. Questo ha permesso agli sviluppatori di operare su un formato di dati familiare e di concentrarsi sul quadro generale.

Con la diffusione dell'accesso alle QPU e con lo sviluppo di un maggior numero di algoritmi quantistici, è emersa nuovamente la necessità di un'astrazione di livello superiore. In risposta, Qiskit ha introdotto l'interfaccia *primitives*, ottimizzata per due compiti fondamentali nello sviluppo di algoritmi quantistici:

- la stima del valore di aspettativa (**Estimator**)
- e il campionamento dei circuiti (**Sampler**).

3: Execution

Primitive

Una primitiva è la più piccola istruzione di elaborazione, il più semplice blocco di costruzione da cui si può creare qualcosa di utile per un determinato livello di astrazione da cui si può creare qualcosa di utile per un determinato livello di astrazione.

In breve, il modello computazionale introdotto da **Qiskit primitives** avvicina la programmazione quantistica a quella classica, dove l'attenzione si concentra meno sui dettagli hardware e più sui risultati alla programmazione classica, dove l'attenzione si concentra meno sui dettagli dell'hardware e più sui risultati che si vogliono ottenere risultati che si vogliono ottenere.

3: Execution

Esistono due tipi di Qiskit primitives : le classi base e le loro implementazioni. I provider (come Qiskit Runtime) possono utilizzare queste classi base per derivare le proprie implementazioni di Sampler e Estimator.

Classi base - *BaseEstimatorV2 e BaseSamplerV2*

Implementazioni - Le primitive di Qiskit Runtime (*EstimatorV2 e SamplerV2*) forniscono un'implementazione più sofisticata (ad esempio, includendo la mitigazione degli errori) come servizio basato su cloud. Questa implementazione delle primitive di base viene utilizzata per accedere all'hardware di IBM Quantum. Vi si accede attraverso il sito IBM Qiskit Runtime.

StatevectorEstimator e StatevectorSampler - Implementazioni di riferimento delle primitive che utilizzano il simulatore integrato in Qiskit.
BackendEstimatorV2 e BackendSamplerV2 - Devono essere inizializzate con un argomento aggiuntivo backend per selezionare il computer quantistico su cui eseguirle.

3: Execution

```
estimator = EstimatorV2(mode=backend)
estimator_job = estimator.run(
  [
    (circuit1, observable1, param_values1),
    (circuit2, observable2, param_values2),
  ]
)
```

```
sampler = SamplerV2(mode=backend)
sampler_job = sampler.run(
  [
    (circuit1, param_values1, shots1),
    (circuit2, param_values2, shots2),
  ]
)
```

3: Execution

INPUT

Queste primitive consentono di definire in modo efficiente i carichi di lavoro vettoriali utilizzando una struttura di dati nota come **Primitive Unified Bloc (PUB)**. Questi PUB sono l'unità di lavoro fondamentale di cui una QPU ha bisogno per eseguire questi carichi di lavoro. Vengono utilizzati come input per il metodo run() per le primitive Sampler e Estimator, che eseguono il carico di lavoro definito come un lavoro.

Quando si invoca il metodo run() il principale argomento richiesto è un list di una o più tuple, una per ogni circuito eseguito dalla primitiva. Ciascuna di queste tuple è considerata un PUB, e gli elementi richiesti di ciascuna tupla dell'elenco dipendono dalla primitiva utilizzata.

(circuito, valori_parametri, shots)

4: Results

OUTPUT

Una volta che uno o più PUB sono stati inviati a una QPU per l'esecuzione e un lavoro è stato completato con successo, i dati vengono restituiti come oggetto *PrimitiveResult* oggetto contenitore a cui si accede chiamando il metodo *RuntimeJobV2.result()* . *PrimitiveResult* contiene un elenco iterabile di oggetti che contengono i risultati dell'esecuzione per ogni PubResult che contiene i risultati dell'esecuzione per ogni PUB.

Ciascuno di questi oggetti *PubResult* possiede un attributo data e uno metadata . L'attributo data è un attributo personalizzato *DataBin* che contiene i valori di misurazione effettivi, le deviazioni standard e così via.

Nel frattempo, l'attributo metadata contiene informazioni sulle opzioni di runtime e di mitigazione degli errori utilizzate.

4: Results

Estimator OUTPUT

```
└── PrimitiveResult
    ├── PubResult[0]
    │   ├── metadata
    │   └── data  ## In the form of a DataBin object
    │       ├── evs
    │       │   └── List of estimated expectation values in the shape
    │       │         specified by the first pub
    │       └── stds
    │           └── List of calculated standard deviations in the
    │                 same shape as above
    ├── PubResult[1]
    │   ├── metadata
    │   └── data  ## In the form of a DataBin object
    │       ├── evs
    │       │   └── List of estimated expectation values in the shape
    │       │        specified by the second pub
    │       └── stds
    │           └── List of calculated standard deviations in the
    │                same shape as above
    ├── ...
    ├── ...
    └── ...
```

Sampler OUTPUT

```
└── PrimitiveResult
    ├── PubResult[0]
    │   ├── metadata
    │   └── data  ## In the form of a DataBin object
    │       ├── NAME_OF_CLASSICAL_REGISTER
    │       │   └── BitArray of count data (default is 'meas')
    │       │
    │       └── NAME_OF_ANOTHER_CLASSICAL_REGISTER
    │           └── BitArray of count data (exists only if more than one
    │                 ClassicalRegister was specified in the circuit)
    ├── PubResult[1]
    │   ├── metadata
    │   └── data  ## In the form of a DataBin object
    │       └── NAME_OF_CLASSICAL_REGISTER
    │           └── BitArray of count data for second pub
    ├── ...
    ├── ...
    └── ...
```

4: Results

OUTPUT

Quando un processo Sampler viene completato con successo, l'oggetto PrimitiveResult restituito contiene un elenco di SamplerPubResults, uno per ogni PUB e. I contenitori di dati di questi SamplerPubResult oggetti sono oggetti simili a dizionari che contengono uno BitArray per ClassicalRegister ogni circuito.

La classe BitArray è un contenitore di dati ordinati. In dettaglio, memorizza le stringhe di bit campionate come byte all'interno di un array bidimensionale.

Come primo esempio, esaminiamo il seguente circuito a dieci qubit.

4: Results

OUTPUT

```
circuit = QuantumCircuit(10)
circuit.h(0)
circuit.cx(range(0, 9), range(1, 10))
circuit.measure_all()
transpiled_circuit = pm.run(circuit)
sampler = Sampler(mode=backend)
job = sampler.run([transpiled_circuit])
result = job.result()
# the data bin contains one BitArray
data = result[0].data
print(f"Databin: {data}\n")
# to access the BitArray, use the key "meas", which is the default name of
# the classical register when this is added by the `measure_all` method
array = data.meas
print(f"BitArray: {array}\n")
print(f"The shape of register `meas` is {data.meas.array.shape}.\n")
print(f"The bytes in register `alpha`, shot by shot:\n{data.meas.array}\n")
# optionally, convert away from the native BitArray format to a dictionary format
counts = data.meas.get_counts()
print(f"Counts: {counts}")
```

```
Databin: DataBin(meas=BitArray(<shape=(), num_shots=4096, num_bits=10>))

BitArray: BitArray(<shape=(), num_shots=4096, num_bits=10>)

The shape of register `meas` is (4096, 2).

The bytes in register `alpha`, shot by shot:
[[  0 128]
 [  0 128]
 [  3 255]
 ...
 [  3 255]
 [  2   1]
 [  3 128]]
```

```
Counts: {'0010000000': 79, '1111111111': 999, '1111111010': 11, '1111000000': 11,
```

Esercizio

1.Creare un circuito a 2 qubit che:

1. Applichi una porta H sul qubit 0
2. Applichi una CX tra qubit 0 (controllo) e qubit 1 (target)
3. Applichi una rotazione Ry(θ) sul qubit 0
4. Misuri entrambi i qubit

2.Definire un parametro θ usando Parameter.

3.Creare un **PUB** contenente:

1. Il circuito
2. Un insieme di valori per θ (es. [0, π/4, π/2, π])
3. Il numero di shot (es. 1000)

4.Transpilare il circuito per il backend scelto.

5.Eseguire il PUB usando il Sampler.

6.Stampare:

1. Le probabilità di output
2. I bitstring misurati

Simulazione esatta e rumorosa

La simulazione esatta con Qiskit primitives mostra come utilizzare le primitive di riferimento incluse in Qiskit per eseguire la simulazione esatta di circuiti quantistici. (es. **StateVector**)

I processori quantistici attualmente esistenti sono affetti da errori, o rumore, per cui i risultati di una simulazione esatta non riflettono necessariamente i risultati che ci si aspetterebbe eseguendo i circuiti su hardware reale.

Mentre le primitive di riferimento di Qiskit non supportano il rumore di modellazione, **Qiskit Aer** include implementazioni delle primitive che supportano il rumore di modellazione.

Qiskit Aer è un simulatore di circuiti quantistici ad alte prestazioni che può essere utilizzato al posto delle primitive di riferimento per ottenere migliori prestazioni e maggiori funzionalità.

https://qiskit.github.io/qiskit-aer/index.html

Simulazione esatta e rumorosa

```
from qiskit.circuit.library import efficient_su2
from qiskit.quantum_info import SparsePauliOp
from qiskit.transpiler import generate_preset_pass_manager
from qiskit_aer import AerSimulator
from qiskit_aer.primitives import EstimatorV2 as Estimator
from qiskit_aer.noise import NoiseModel, depolarizing_error

n_qubits = 8
circuit = efficient_su2(n_qubits)
circuit.draw("mpl")
observable = SparsePauliOp("Z" * n_qubits)
params = [0.1] * circuit.num_parameters

exact_estimator = Estimator()
# The circuit needs to be transpiled to the AerSimulator target
pass_manager = generate_preset_pass_manager(3, AerSimulator())
isa_circuit = pass_manager.run(circuit)
pub = (isa_circuit, observable, params)
job = exact_estimator.run([pub])
 result = job.result()
pub_result = result[0]
exact_value = float(pub_result.data.evs)
exact_value

-----

noise_model = NoiseModel()
cx_depolarizing_prob = 0.02
noise_model.add_all_qubit_quantum_error(
   depolarizing_error(cx_depolarizing_prob, 2), ["cx"])

noisy_estimator = Estimator(
   options=dict(backend_options=dict(noise_model=noise_model)))
job = noisy_estimator.run([pub])
result = job.result()
pub_result = result[0]
noisy_value = float(pub_result.data.evs)
noisy_value
```

Esercizio

Eseguire il circuito precedente con il Sampler e mostrare i risultati nelle diverse visualizzazioni

Costruire modelli di rumore

Il modulo Qiskit Aer noise contiene le classi Python per costruire modelli di rumore personalizzati per la simulazione. Le classi principali sono tre:

- La classe *NoiseModel* che memorizza un modello di rumore utilizzato per la simulazione del rumore.
- La classe *QuantumError* che descrive gli errori dei gate.
- La classe ReadoutError che descrive gli errori di lettura classici.

È possibile inizializzare un modello di rumore con i parametri impostati dai dati di calibrazione più recenti per un backend fisico. In questo modo si otterrà un modello di rumore che approssima gli errori che si incontrano quando si usa quel backend.

Se si desidera avere un controllo più dettagliato sui parametri del modello di rumore, è necessario creare il proprio modello di rumore

Problema di versioni

Attenzione alle versioni della documentazione e dei tutorial, molto spesso sono compatibili per versioni vecchie di qiskit.

https://qiskit.github.io/qiskit-aer/
https://github.com/Qiskit/qiskit-aer

Qiskit IBM Runtime

```
pip install qiskit-ibm-runtime

from qiskit_ibm_runtime import QiskitRuntimeService

QiskitRuntimeService.save_account(
token="<your-api-key>",
instance="<CRN>", # Optional
)

service = QiskitRuntimeService()
backend = service.least_busy(simulator=False,
operational=True)
#real_backend = service.backend("ibm_fez")

pm =
generate_preset_pass_manager(backend=backend,
optimization_level=1)
```

Simulazione

```
from qiskit_ibm_runtime.fake_provider import
FakeBelemV2
backend = FakeBelemV2()
```

https://quantum.cloud.ibm.com/

Fake providers

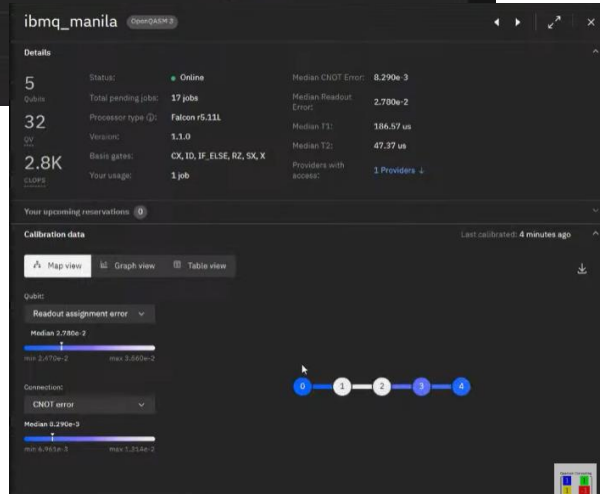
- FakeAlgiers
- FakeAlmadenV2
- FakeArmonkV2
- FakeAthensV2
- FakeAuckland
- FakeBelemV2
- FakeBoeblingenV2
- FakeBogotaV2
- FakeBrisbane
- FakeBrooklynV2
- FakeBurlingtonV2
- FakeCairoV2

I **fake backend** sono costruiti per imitare i comportamenti dei sistemi IBM Quantum utilizzando le istantanee del sistema.

Le istantanee del sistema contengono informazioni importanti sul sistema quantistico, come la mappa di accoppiamento, le porte di base, le proprietà dei qubit (T1, T2, il tasso di errore, ecc.) che sono utili per testare il transpiler ed eseguire simulazioni rumorose del sistema.

IBM quantum cloud

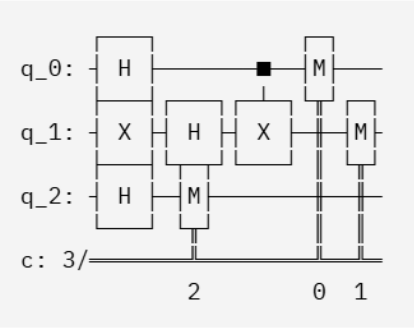
Name	Qubits	QV	CLOPS	Status	Total pending jobs	Processor type	Plan	Features
ibm_nairobi	7	32	2.6K	Online	27	Falcon r5.11H	open	OpenQASM 3
ibm_oslo	7	32	2.6K	Online	21	Falcon r5.11H	open	OpenQASM 3
ibmq_manila	5	32	2.8K	Online	17	Falcon r5.11L	open	OpenQASM 3
ibmq_quito	5	16	2.5K	Online	27	Falcon r4T	open	
ibmq_belem	5	16	2.5K	Online	13	Falcon r4T		
ibmq_lima	5	8	2.7K	Online	139	Falcon r4T		



Visualizzazione

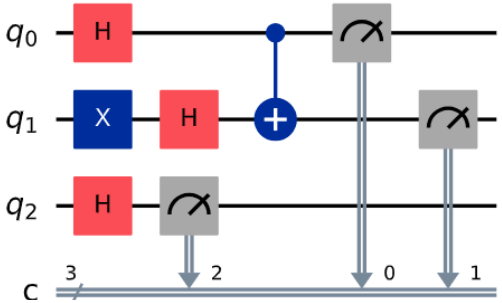
```
circuit.draw()
```

Output:



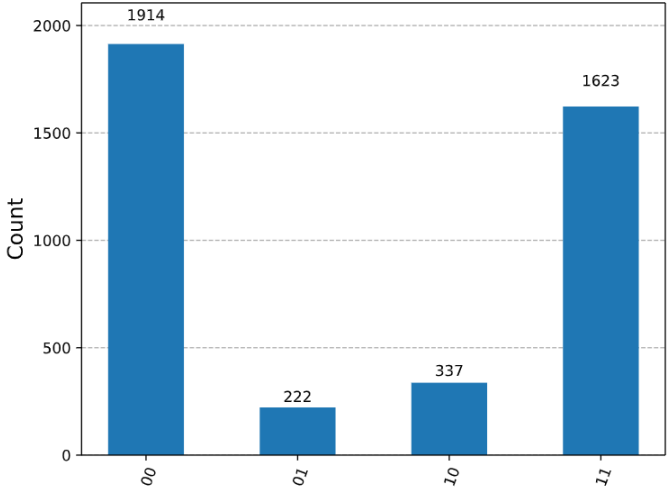

```
# Save as an image using the Matplotlib drawer
circuit.draw(output="mpl", filename="circuit-mpl.jpeg")
```

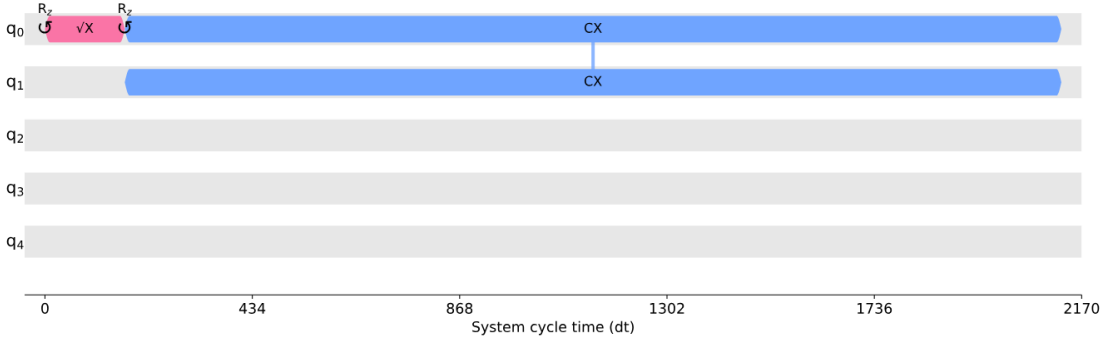
Output:




```
plot_histogram(result[0].data.meas.get_counts())
```

Output:






```
from qiskit.visualization.timeline import draw

…

draw(isa_circuit, target=backend.target)
```

Esercizio

Eseguire un circuito a scelta su simulatore ideale e simulatore rumoroso e comparare i risultati.

Modello Digitale vs Modello Analogico

Modello a Circuiti (Gate-Based)
Computazione tramite sequenza di porte unitarie

Evoluzione discreta: $|\psi out\rangle = Un \ldots U2U1|\psi in\rangle$

Ogni operazione è una porta (H, CNOT, RX, ...)

È il modello usato in IBM Quantum e nei computer quantistici universali

Modello Analogico (Hamiltonian-Based)

Il sistema evolve secondo una Hamiltoniana H

Evoluzione continua nel tempo: $|\psi(t)\rangle = e^{-iHt}|\psi(0)\rangle$

Non si applicano "porte", ma si lascia evolvere il sistema

Modello Analogico

Una **Hamiltoniana** è un operatore hermitiano che descrive: energia del sistema, interazioni tra qubit, campi esterni.

L'evoluzione è: $U(t)=e^{-iHt}$

Nel modello digitale possiamo approssimarla (Trotterizzazione): $e^{-i(A+B)t} \approx e^{-iAt} e^{-iBt}$

I circuiti possono simulare evoluzioni analogiche.

In Qiskit possiamo definire Hamiltoniane con operatori di Pauli, costruire l'operatore di evoluzione e simulare l'evoluzione temporale.

Modello Analogico

```
import numpy as np
from qiskit import QuantumCircuit
from qiskit.quantum_info import SparsePauliOp
from qiskit.circuit.library import PauliEvolutionGate
from qiskit.primitives import StatevectorSampler

H = SparsePauliOp.from_list([("ZZ", 1.0)])
t = 1.0
evolution = PauliEvolutionGate(H, time=t)

qc = QuantumCircuit(2)
qc.h(0)
qc.h(1)
qc.append(evolution, [0,1])
qc.measure_all()

sampler = StatevectorSampler()
result = sampler.run([(qc, {}, 1000)]).result()

print(result[0].data.meas.get_counts())
```

SparsePauliOp → descrive l'energia del Sistema
PauliEvolutionGate → implementa l'evoluzione fisica

Molti algoritmi usano Hamiltoniane:

- VQE
- Simulazione quantistica
- QAOA
- Quantum annealing

PENNYLANE

PennyLane (https://pennylane.ai/) is a cross-platform Python library for programming quantum computers. Its differentiable programming paradigm enables the execution and training of quantum programs on various backends.

PennyLane connects quantum computing with powerful machine learning frameworks like NumPy's autograd, JAX, and PyTorch, making them quantum-aware.

Program	Integrate with	Master
quantum computers	machine learning	quantum chemistry

The library comes with default simulator devices, but is well-integrated with external software and hardware to run quantum circuits—such as IBM's Qiskit, or Google's Cirq, Rigetti's Forest, or Xanadu's Strawberry Fields.

PENNYLANE

Device

`default.qubit`

A simple Python state vector simulator of qubit-based quantum circuits.

This is PennyLane's most featureful device, ideal for learning and prototyping.

`lightning.qubit`

A performant state vector simulator of quantum circuits.

This device is well-suited for 20+ qubit workflows, offering jit/qjit compatibility and more.

`qiskit.remote`

An entry point in PennyLane to access any Qiskit backend.

This device lets you access any device available through qiskit_ibm_runtime.

`default.mixed`

A simple mixed state simulator of qubit-based quantum circuits.

This device supports versatile noise models that are compatible with ML frameworks like JAX and Torch.

PENNYLANE

```python
import pennylane as qml

from jax import numpy as np

import jax

dev1 = qml.device("lightning.qubit", wires=1)


@qml.qnode(dev1)

def circuit(params):

    qml.RX(params[0], wires=0)

    qml.RY(params[1], wires=0)

    return qml.expval(qml.PauliZ(0))

params = np.array([0.54, 0.12])

print(circuit(params))
```

- name: the name of the device to be loaded
- wires: the number of subsystems to initialize the device with (qubits)

Qnode è un circuito quantum legato ad un particolare device

Esercizio

In questo esercizio costruite un circuito quantistico a **3 qubit** usando PennyLane. Il circuito sarà parametrico e diviso in più "layer":

1. **Layer di rotazioni singole:** applicate su ciascun qubit una rotazione parametrica (RX, RY, RZ) con parametri $\theta_1, \theta_2, \theta_3$.
2. **Layer di entanglement:** collegate i qubit con CNOT per creare correlazioni tra loro.
3. **Secondo blocco parametrico:** applicate altre rotazioni sui qubit con nuovi parametri $\theta_4, \theta_5, \theta_6$.

Dopo aver definito il circuito come **QNode**, dovrete valutare:

- i valori attesi di PauliZ per ciascun qubit
- le probabilità complete sul sistema a 3 qubit (qml.probs)
- Provate a calcolare i risultati con:
- tutti i parametri θ a 0
- tutti i parametri θ a π/4
- un vettore di parametri a scelta

Infine, osservate come cambiano le probabilità e i valori attesi, e commentate l'effetto dell'entanglement e dei parametri sul circuito.